

GUÍA DE TRABAJO — ESTUDIANTE
SEMANA 2: ESPECIFICACIÓN FORMAL DE SEGURIDAD Y LOGIN SEGURO
Programación Segura

Nombre del estudiante: Ana Concepción Asto Arotinco
Grupo / Sección: DD281
Fecha: 10 de junio de 2026

SECCIÓN A — RECUPERACIÓN DE APRENDIZAJES (Semana 1)

A.1 La Triada CIA aplicada al Login

Pilar CIA	¿Cómo se manifiesta en un sistema de login?	Ejemplo concreto
Confidencialidad	Las credenciales (usuario/contraseña) solo son visibles para el usuario y el sistema. Nadie más puede leer o interceptar esos datos.	Las contraseñas se transmiten cifradas por TLS y se almacenan como hash bcrypt, nunca en texto plano.
Integridad	Los datos de autenticación no son alterados en tránsito ni en almacenamiento; el sistema verifica que la contraseña no haya sido modificada.	Se usa bcrypt para asegurar que el hash almacenado no puede ser revertido ni manipulado para crear colisiones útiles.
Disponibilidad	El sistema de login debe estar operativo cuando el usuario legítimo lo necesite, sin caídas provocadas por ataques.	Protección contra ataques de denegación de servicio (DoS) limitando la tasa de intentos y usando bloqueo progresivo de cuentas.

A.2 OWASP Top 10 — Relación con Autenticación

#	Vulnerabilidad OWASP	¿Relacionada con login?	¿Por qué?
A01	Broken Access Control	SI	Un login deficiente permite que usuarios no autorizados accedan a recursos que no les corresponden
A02	Cryptographic Failures	SI	Almacenar contraseñas en texto plano o con hashes débiles (MD5, SHA-1) expone las credenciales.
A03	Injection	SI	Si la query SQL no usa parámetros preparados, un atacante puede inyectar SQL en el campo usuario o contraseña para bypassar el login.
A04	Insecure Design	SI	Un diseño sin política de bloqueo, sin MFA o sin especificación formal de seguridad es inherentemente inseguro desde la arquitectura
A07	Identification/Auth	SI	Es la categoría directa: contraseñas débiles, ausencia de MFA, sesiones que no expiran y falta de protección contra fuerza bruta

A.3 Principio de Mínimo Privilegio

¿Cómo aplicarías el principio de Mínimo Privilegio a un sistema de autenticación? Escribe al menos 3 aplicaciones concretas:

1. El usuario autenticado con rol 'estudiante' solo puede acceder a sus propios registros y recursos académicos; no tiene acceso a las cuentas de otros estudiantes ni a módulos administrativos.
2. El proceso de la aplicación que consulta la base de datos usa una cuenta de BD con permisos únicamente de SELECT sobre la tabla de usuarios, sin privilegios de DELETE, DROP ni acceso a otras tablas.
3. Los administradores del sistema tienen acceso al panel de gestión solo desde redes internas (VPN); incluso con credenciales válidas, el acceso desde IPs externas no autorizadas es denegado automáticamente.

SECCIÓN B — ACTIVIDAD DIAGNÓSTICA

B.1 ¿Qué hace que un login sea inseguro?

- Almacenar contraseñas en texto plano o con hashes débiles como MD5.
- No usar TLS/HTTPS, transmitiendo credenciales en texto claro.
- No limitar el número de intentos de login (sin protección ante fuerza bruta).
- Usar consultas SQL concatenadas directamente (SQL Injection).
- Mostrar mensajes de error que revelan si el usuario existe o no.
- No implementar autenticación multifactor (MFA) en sistemas críticos.

B.2 Preguntas de Diagnóstico

Responde con lo que ya sabes (marca la opción más correcta):

****B.2.1**** ¿Cómo se deben almacenar las contraseñas en una base de datos?

- ☐ a) En texto plano para facilitar la recuperación
- ☐ b) Cifradas con AES-256 para poder descifrarlas si el usuario las olvida
- x c) **Como hash unidireccional con sal aleatoria**
- ☐ d) Codificadas en Base64 para "ofuscarlas"

****B.2.2**** ¿Qué versión de TLS/SSL deben usar los servidores web en 2024?

- ☐ a) SSL 3.0 — es la versión estándar
- ☐ b) TLS 1.0 — compatible con todos los dispositivos
- x c) **TLS 1.2 mínimo, preferiblemente TLS 1.3**
- ☐ d) La versión no importa, cualquier SSL es suficiente

****B.2.3**** ¿Qué es un certificado SSL autofirmado (self-signed)?

- ☐ a) Un certificado gratuito de Let's Encrypt
- x b) **Un certificado creado por uno mismo sin validación de una CA externa**
- ☐ c) Un certificado de mayor seguridad que el emitido por una CA
- ☐ d) El tipo de certificado requerido en producción

SECCIÓN C — CONCEPTOS CLAVE DE LA SESIÓN

C.1 Especificación Formal de Seguridad

Una especificación formal de seguridad es un documento técnico preciso y verificable que define, sin ambigüedad, quién puede acceder a qué recursos, bajo qué condiciones, mediante qué mecanismos y qué ocurre cuando se detecta una violación. A diferencia de las políticas de seguridad genéricas, una especificación formal puede ser auditada, implementada directamente en código y verificada mediante pruebas.

C.1.1 ¿Cuál es la diferencia entre estas dos "especificaciones"? ¿Por qué una es formal y la otra no?

	Ejemplo A	Ejemplo B
Texto	"El login debe ser seguro"	"Las contraseñas se almacenarán como hash bcrypt con factor de coste 12. Máximo 5 intentos antes de bloqueo de 15 min."
¿Por qué es o no es formal?	NO es formal: es subjetiva, no verificable, no define ningún mecanismo concreto ni criterio de cumplimiento.	SÍ es formal: especifica el algoritmo exacto (bcrypt), el parámetro cuantificable (factor 12), el límite de intentos (5) y el tiempo de bloqueo (15 min). Puede ser implementada y auditada directamente.

C.1.2 — Los 7 componentes de una especificación formal:

#	Componente	¿Qué define?
1	Activos	Los datos o recursos que deben protegerse: credenciales de usuario, tokens de sesión, base de datos de contraseñas.
2	Sujetos	Las entidades que interactúan con el sistema: usuario final, administrador, proceso automatizado, atacante potencial.
3	Objetos	Los recursos a los que se controla el acceso: formulario de login, endpoint de autenticación, tabla de usuarios en BD.
4	Operaciones	Las acciones permitidas o denegadas: leer, escribir, autenticar, modificar contraseña, bloquear cuenta.
5	Condiciones	Las circunstancias bajo las cuales se permite o deniega una operación: máximo de intentos, horario permitido, IP de origen.
6	Mecanismos	Las soluciones técnicas que implementan la política: bcrypt, TLS 1.3, prepared statements, rate limiting, HSTS.
7	Respuesta ante violación	Las acciones automáticas al detectar un ataque: bloqueo de cuenta, alerta al administrador, registro en log, IP ban temporal.

C.2 Preguntas de Selección Múltiple — Respuestas correctas:

****C.2.1**** ¿Cuál es el problema de almacenar contraseñas con SHA-1 SIN SAL?

- ☐ a) SHA-1 no produce un hash — produce texto cifrado
- ☐ b) SHA-1 es reversible — se puede obtener la contraseña original
- X ☒ c) Los atacantes pueden usar tablas rainbow precomputadas para romper el hash
- ☐ d) SHA-1 produce hashes demasiado cortos para ser seguros

****C.2.2**** ¿Qué ventaja fundamental tiene bcrypt sobre SHA-256 para almacenar contraseñas?

- ☐ a) Bcrypt produce hashes más largos que SHA-256
- X ☒ b) Bcrypt incluye automáticamente sal aleatoria y es intencionalmente lento
- ☐ c) Bcrypt es un algoritmo de cifrado, no de hash
- ☐ d) Bcrypt es más rápido que SHA-256, mejorando el rendimiento del login

****C.2.3**** ¿Qué es la "sal" (salt) en el contexto del hashing de contraseñas?

- ☐ a) Un algoritmo de cifrado adicional aplicado al hash
- ☐ b) La clave secreta usada para cifrar el hash antes de almacenarlo
- X ☒ c) Un valor aleatorio único por usuario que se concatena a la contraseña antes de hashear
- ☐ d) El factor de coste que determina cuántas rondas de hashing se ejecutan

****C.2.4**** ¿Por qué es un error de seguridad grave que el formulario de login use el método HTTP GET?

- ☐ a) Porque GET no puede transportar datos de texto
- X ☒ b) Porque los parámetros GET viajan en la URL y quedan en logs del servidor y en el historial del navegador
- ☐ c) Porque GET es más lento que POST para transferir datos
- ☐ d) Porque GET no cifra los datos antes de enviarlos

****C.2.5**** ¿Qué es Perfect Forward Secrecy (PFS) en TLS?

- ☐ a) Un mecanismo que cifra el certificado del servidor con una segunda clave
- ☐ b) La capacidad del servidor de descifrar tráfico pasado si se compromete la clave privada
- ☒ c) El uso de claves de sesión efímeras para que el compromiso de la clave privada del servidor no permita descifrar tráfico pasado
- ☐ d) La verificación automática de que el certificado SSL no ha expirado

****C.2.6**** ¿Cuál de los siguientes es el estándar de hash de contraseñas más recomendado hoy?

- ☐ a) MD5 con sal de 16 bytes
- ☐ b) SHA-512 sin sal
- ☒ c) bcrypt (factor 12+) o argon2id
- ☐ d) AES-256 con clave de 32 bytes

****C.2.7**** ¿Cuál de las siguientes configuraciones de servidor web es correcta en relación a SSL?

- ☐ a) Habilitar SSL 3.0, TLS 1.0, TLS 1.1 y TLS 1.2 para máxima compatibilidad
- ☐ b) Usar solo TLS 1.3 y deshabilitar todas las versiones anteriores
- ☐ c) Deshabilitar SSL 2.0 y SSL 3.0, mantener TLS 1.0, 1.1, 1.2 y 1.3
- ☒ d) Usar TLS 1.2 y TLS 1.3, deshabilitar versiones anteriores

****C.2.8**** ¿Qué es CGI (Common Gateway Interface)?

- ☐ a) Un framework de Python para desarrollo web seguro
- ☒ b) Un protocolo estándar que define cómo un servidor web pasa solicitudes a programas externos para generar respuestas dinámicas
- ☐ c) Una librería de JavaScript para crear formularios de login
- ☐ d) Un tipo de certificado SSL para servidores compartidos

****C.2.9**** Un atacante ejecuta el siguiente input en el campo de usuario de un login CGI inseguro: `admin' --`. ¿Qué tipo de ataque es este y qué efecto tendría?

- ☐ a) XSS — inyecta código JavaScript en la página
- X b) **SQL Injection** — el `'--`` cierra la query y comenta el resto, posiblemente **bypassando la verificación de contraseña**
- ☐ c) CSRF — falsifica una solicitud de otro dominio
- ☐ d) Path Traversal — intenta acceder a archivos del sistema

****C.2.10**** ¿Cuál es el propósito del header HTTP `Strict-Transport-Security`?

- ☐ a) Obliga al servidor a responder solo con JSON
- X b) **Le indica al navegador que siempre use HTTPS para ese dominio, incluso si el usuario escribe HTTP**
- ☐ c) Restringe el origen de las solicitudes al dominio del servidor
- ☐ d) Cifra automáticamente todos los cookies del servidor

C.3 Preguntas de Completar

Completa los espacios en blanco con la palabra o frase correcta:

- C.3.1 El proceso de almacenamiento de contraseñas usa **hash** (no cifrado), porque es un proceso **unidireccional** que no permite obtener el dato original.
- C.3.2 La "sal" en bcrypt es un valor **aleatorio** y **único por usuario** por usuario que elimina la posibilidad de usar **tablas rainbow** precomputadas.
- C.3.3 TLS 1.3 hace obligatorio el uso de **PFS (Perfect Forward Secrecy)**, (siglas), lo que significa que si la clave privada del servidor se compromete, el tráfico **pasado** no puede ser descifrado.
- C.3.4 En CGI, los datos del formulario POST se reciben a través de la **entrada** estándar del script, mientras que los parámetros GET llegan en la variable de entorno **QUERY_STRING**.
- C.3.5 El código de respuesta HTTP que se debe usar para redirigir permanentemente HTTP a HTTPS es el **301 Moved Permanently** (o 302 para temporal).
- C.3.6 El principio de seguridad que dice que cada usuario o proceso debe tener solo los permisos mínimos necesarios se llama **Mínimo Privilegio (Least Privilege)**.
- C.3.7 Un certificado SSL **self-signed** (autofirmado) es apropiado para **entornos de desarrollo** y **laboratorios**, pero NO para **producción** porque los navegadores muestran una advertencia de seguridad.
- C.3.8 La organización OWASP clasifica como A07 los fallos de **Identificación** y **Autenticación**, que incluyen contraseñas débiles, ausencia de MFA y sesiones que no expiran.

SECCIÓN D — PREGUNTAS DE ANÁLISIS

D.1 Lee el siguiente fragmento de código Python CGI. Identifica y describe 5 vulnerabilidades de seguridad específicas que contiene:

```
#!/usr/bin/env python3
import cgi
import pymysql

print("Content-Type: text/html\n")

form = cgi.FieldStorage()
user = form.getvalue("user")
pwd = form.getvalue("pwd")

conn = pymysql.connect(host="db.empresa.com", user="admin",
                       password="Admin@2024!", database="empresa")
cursor = conn.cursor()
sql = f"SELECT * FROM empleados WHERE usuario='{user}' AND clave='{pwd}'"
cursor.execute(sql)
row = cursor.fetchone()

if row:
    print(f"<h1>Acceso concedido a: {user}</h1>")
    print(f"<p>Datos del empleado: {row}</p>")
else:
    print(f"<h1>Acceso denegado. Usuario: {user} no existe.</h1>")
conn.close()
```

#	Vulnerabilidad	Riesgo	Corrección
1	SQL Injection (línea sql = f"...{user}...")	Un atacante puede inyectar SQL para bypassar la autenticación, extraer todos los usuarios o destruir la BD.	Usar prepared statements con parámetros: cursor.execute("SELECT... WHERE usuario=?", (user,))
2	Credenciales de BD hardcodeadas (password="Admin@2024!")	Cualquiera que lea el código fuente (repositorio, backup, log) obtiene acceso total a la base de datos.	Leer credenciales desde variables de entorno o archivos de configuración fuera del repositorio (python-dotenv, AWS Secrets Manager).
3	XSS reflejado (f"Acceso concedido a: {user}")	Si user contiene <script>alert(1)</script>, se ejecuta JavaScript en el navegador de la víctima.	Escapar toda entrada del usuario con html.escape(user) antes de incluirla en HTML.
4	Revelación de datos sensibles (f"Datos del empleado: {row}")	Expone todos los campos del registro de BD, incluyendo potencialmente el hash de contraseña, roles y datos personales.	Mostrar únicamente los datos mínimos necesarios. Nunca reflejar el contenido de filas de BD directamente.

5	User enumeration ("El usuario X no existe")	El atacante puede enumerar usuarios válidos: si el mensaje es diferente para "usuario no existe" vs "contraseña incorrecta", puede construir una lista de usuarios del sistema.	Usar siempre el mismo mensaje genérico: "Credenciales incorrectas." independientemente de si el usuario existe o no.
---	---	---	--

D.2 Analiza la siguiente política de contraseñas de una empresa e identifica qué está bien y qué está mal según las guías NIST SP 800-63B:

"Política de contraseñas de TechCorp SA: Las contraseñas deben tener exactamente 8 caracteres. Deben incluir al menos una letra mayúscula, una minúscula, un número y un símbolo. Las contraseñas deben cambiarse obligatoriamente cada 30 días. El sistema almacena las últimas 3 contraseñas para no repetirlas. El campo acepta cualquier combinación de caracteres ASCII."

Elemento	¿Correcto o incorrecto?	¿Por qué?
Exactamente 8 caracteres	INCORRECTO	NIST recomienda un MÍNIMO de 8 caracteres, pero con un máximo muy alto (al menos 64). Fijar exactamente 8 impide contraseñas más largas y seguras como frases de contraseña.
Cambio obligatorio cada 30 días	INCORRECTO	NIST SP 800-63B desaconseja explícitamente el cambio periódico forzado sin evidencia de compromiso. Forzar cambios frecuentes lleva a contraseñas predecibles (ej: "Enero2026!").
Historial de 3 passwords	PARCIALMENTE CORRECTO	Evitar la reutilización inmediata es buena práctica. NIST lo acepta. Sin embargo, el número 3 es bajo; NIST sugiere verificar contra listas de contraseñas comprometidas (HaveIBeenPwned).
Complejidad obligatoria (mayus+minus+num+simb)	INCORRECTO	NIST desaconseja los requisitos de complejidad obligatoria porque producen contraseñas débiles predecibles (ej: "Abcd1!"). Prefiere contraseñas largas y verificación contra listas de contraseñas comprometidas.

D.1 Escenario Fintech — Startup de préstamos personales (SBS)

D.1.1 — Estándares internacionales relevantes:

• PCI DSS (Payment Card Industry Data Security Standard): obligatorio si la app procesa pagos con tarjeta. Exige autenticación multifactor, cifrado en tránsito y en reposo, y gestión segura de contraseñas.
• ISO/IEC 27001:2022 — Sistema de Gestión de Seguridad de la Información: marco completo para gestión de riesgos, incluyendo controles de acceso y autenticación.
• NIST SP 800-63B — Digital Identity Guidelines: estándar técnico para autenticación, gestión de contraseñas y uso de MFA, muy referenciado por reguladores en Latinoamérica.

- Resolución SBS N° 504-2021 (Reglamento de Gobierno del Sistema de Información): norma local peruana que exige controles de seguridad para entidades supervisadas por la SBS.

D.1.2 — Especificación formal de seguridad para el módulo de login:

Componente	Especificación
Activos a proteger	Credenciales de usuario (usuario + contraseña), tokens de sesión JWT, base de datos de clientes financieros, historial de intentos de acceso.
Sujetos (roles)	Cliente (acceso solo a su cuenta), Asesor Financiero (acceso a carteras asignadas), Administrador (gestión del sistema), Sistema de Auditoría (solo lectura de logs).
Objetos (recursos)	Endpoint POST /api/auth/login, tabla usuarios en BD PostgreSQL, tabla sesiones_activas, directorio de logs de auditoría.
Operaciones permitidas/denegadas	PERMITIDO: POST /auth/login con credenciales válidas + segundo factor. DENEGADO: GET en endpoint de login, más de 5 intentos en 10 minutos, acceso sin HTTPS, tokens con más de 30 minutos de inactividad.
Condiciones de acceso	Credenciales correctas + OTP por SMS o TOTP (Google Authenticator). IP no en lista de bloqueo. Cuenta no bloqueada. Sesión activa < 30 min inactividad / 8 horas máximo absoluto.
Mecanismos técnicos	bcrypt factor 14 para almacenamiento de contraseñas. TLS 1.3 con HSTS. MFA obligatorio (TOTP RFC 6238). JWT firmado con RS256. Rate limiting: 5 intentos / 10 min por IP y por usuario. Prepared statements para todas las queries. Headers de seguridad: HSTS, X-Frame-Options, CSP.
Respuesta ante violación	Bloqueo automático de cuenta por 30 minutos tras 5 intentos. Alerta inmediata al administrador y al cliente vía email. Log detallado del evento (timestamp, IP, user-agent). Bloqueo de IP tras 20 intentos desde la misma IP en 1 hora. Notificación a equipo de seguridad si se detecta patrón de ataque coordinado.

D.1.3 — Justificación técnica y normativa de bcrypt:

bcrypt es el algoritmo correcto para esta fintech por tres razones fundamentales. Técnicamente: bcrypt incorpora sal aleatoria de forma nativa (eliminando tablas rainbow), usa un algoritmo intencionalmente lento ajustable mediante el factor de coste (lo que hace que un ataque de fuerza bruta sea computacionalmente inviable incluso con hardware GPU moderno), y produce hashes de longitud fija independientemente de la contraseña (previniendo ataques de temporización por longitud). MD5 y SHA-256 sin sal son descartables porque son rápidos —un atacante puede probar miles de millones de hashes por segundo en una GPU— y vulnerables a tablas rainbow. AES se descarta porque es cifrado simétrico reversible: si se compromete la clave de cifrado, se obtienen todas las contraseñas. SHA-256 con sal es mejor que MD5 pero sigue siendo un hash de propósito general diseñado para velocidad, no para protección de contraseñas. Desde el punto de vista normativo: OWASP recomienda explícitamente bcrypt (o Argon2id) como estándares de almacenamiento de contraseñas. El cumplimiento con PCI DSS Requisito 8 exige proteger las credenciales de acceso, y bcrypt es la implementación técnica que satisface ese requisito de manera auditablemente correcta frente a la SBS.

D.2 Caso Adobe — Brecha de 2013

D.2.1 — ¿Cómo Adobe almacenaba las contraseñas?

Adobe almacenaba las contraseñas cifradas con 3DES (Triple DES) en modo ECB, no hasheadas. Este fue un error conceptual grave por dos razones: primero, usaron cifrado reversible en lugar de hash unidireccional, lo que significa que con la clave de descifrado (que también fue comprometida) se podían obtener todas las contraseñas en texto plano. Segundo, el modo ECB cifra cada bloque de datos de forma independiente, lo que significa que contraseñas idénticas producen el mismo texto cifrado —exponiendo qué usuarios tenían la misma contraseña e incluyendo patrones estructurales analizables. Los 153 millones de registros quedaron efectivamente expuestos.

D.2.2 — ¿Por qué el hint de contraseña empeoró el ataque?

Adobe guardaba junto a cada registro un campo de 'pista de contraseña' escrito por el propio usuario. Como miles de usuarios compartían la misma contraseña (ej: '123456') y el mismo texto cifrado, los investigadores de seguridad pudieron correlacionar los hints de miles de usuarios con el mismo hash cifrado y deducir la contraseña original. Por ejemplo: si 100,000 usuarios tenían el mismo valor cifrado y sus hints decían cosas como 'números del 1 al 6', 'seis unos', 'fácil' o 'la más simple', la contraseña quedaba revelada sin necesidad de descifrar nada.

D.2.3 — ¿Qué debió haber hecho Adobe?

Usar bcrypt o PBKDF2 con sal aleatoria única por usuario para almacenar las contraseñas. Nunca almacenar pistas de contraseña en la base de datos. Implementar un programa de Bug Bounty y auditorías de seguridad periódicas. Cifrar la clave de 3DES con un sistema de gestión de claves (KMS) separado e independiente de la BD de usuarios. Implementar segmentación de red para que la base de datos de usuarios no sea accesible directamente desde internet.

SECCIÓN E — ACTIVIDAD COLABORATIVA: ESPECIFICACIÓN FORMAL

Sistema asignado: Portal de Notas Estudiantiles — Universidad Autónoma del Perú

E.1 Especificación Formal — Módulo de Login

Componente	Especificación desarrollada
ACTIVOS (datos protegidos)	• Credenciales de estudiantes (usuario + contraseña hasheada) • Tokens de sesión activos • Notas y expediente académico • Datos personales (DNI, email institucional)
SUJETOS (roles diferenciados)	Rol 1: Estudiante → Permisos: solo lectura de sus propias notas y asistencia. Rol 2: Docente → Permisos: lectura/escritura de notas de sus cursos asignados. Rol 3: Administrador Académico → Permisos: gestión completa del sistema, creación de usuarios.
OBJETOS (recursos controlados)	• Formulario de autenticación en /login • Tabla usuarios en BD • Endpoint de API /api/notas/:id_estudiante
MECANISMOS TÉCNICOS	Algoritmo hash: bcrypt Factor: 12 Protocolo TLS: 1.3 Cipher suites: ECDHE+AESGCM, ECDHE+CHACHA20 Política de contraseñas: longitud mínima 10, complejidad media Política de bloqueo: máx. 5 intentos Duración: 20 min Tipo: temporal progresivo Expiración de sesión: inactividad 30 min / máximo 8 horas MFA: Sí → TOTP (Google Authenticator)
CONDICIONES DE ACCESO	• Acceso solo mediante HTTPS (TLS 1.3) • Contraseña debe pasar bcrypt.checkpw() con hash almacenado • Cuenta no debe estar en estado bloqueado • Sesión no debe haber expirado por inactividad o tiempo máximo

Componente	Especificación desarrollada
LOGGING DE SEGURIDAD	Evento 1: AUTH_SUCCESS — usuario, rol, timestamp, IP Evento 2: AUTH_FAILURE — usuario, intento N/5, timestamp, IP Evento 3: CUENTA_BLOQUEADA — usuario, IP, timestamp, duración del bloqueo
RESPUESTA ANTE VIOLACIÓN	• Bloqueo automático de cuenta por 20 minutos tras 5 intentos fallidos • Registro inmediato en log de auditoría con datos forenses (IP, user-agent, timestamp) • Notificación por email al usuario y al administrador académico

E.2 Justificación de decisiones de diseño:

Decisión técnica	Justificación
Algoritmo de hash: bcrypt factor 12	bcrypt con factor 12 tarda ~250ms por verificación. Esto hace que un ataque de fuerza bruta offline sea inviable incluso con hardware dedicado. El factor 12 es el mínimo recomendado por OWASP para sistemas actuales.
Versión de TLS: 1.3	TLS 1.3 elimina cipher suites inseguros heredados, hace obligatorio PFS y reduce la latencia del handshake. TLS 1.2 y anteriores tienen vulnerabilidades conocidas (BEAST, POODLE, ROBOT).
Política de bloqueo: 5 intentos / 20 min	5 intentos son suficientes para un usuario legítimo que olvidó su contraseña, pero demasiado pocos para un ataque de diccionario automatizado. 20 minutos disuade sin ser tan largo que frustre a usuarios legítimos.
MFA: TOTP obligatorio	Un sistema universitario con datos de notas tiene valor suficiente para justificar el costo de implementar MFA. TOTP no requiere SMS (evita ataques SIM swapping) y es gratuito con apps como Google Authenticator o Authy.

E.3 Caso extremo — Atacante con acceso directo a la base de datos:

Lo que el atacante PODRÍA obtener:

- Nombres de usuario (en texto plano en la columna 'usuario').
- Los hashes bcrypt de las contraseñas (visibles pero no reversibles directamente).
- Metadatos: roles de usuario, fechas de creación, historial de bloqueos.
- Direcciones de email institucional y DNI si están almacenados en la misma tabla.
- El atacante PODRÍA intentar un ataque de fuerza bruta offline contra los hashes bcrypt, aunque sería extremadamente lento por el factor de coste 12.

Lo que el atacante NO PODRÍA obtener (gracias a la especificación):

- Las contraseñas originales en texto plano: bcrypt es unidireccional e irreversible sin fuerza bruta exhaustiva.
- Las contraseñas mediante tablas rainbow: la sal única por usuario en bcrypt invalida completamente este método.
- Tokens de sesión activos (si se almacenan hasheados o en memoria/Redis separado).
- El atacante no puede falsificar una sesión válida solo con los datos de la BD, ya que los tokens JWT están firmados con una clave privada que NO está en la BD.

SECCIÓN F — CIERRE Y METACOGNICIÓN

Vuelve a tu lista de la Sección B.1 (¿qué hace inseguro un inicio de sesión?). Ahora agrega todo lo que descubriste durante la clase:

Nuevas razones identificadas durante la clase:

7. No validar la longitud máxima de los inputs (permite ataques DoS o buffer overflow).
8. Ausencia de headers de seguridad HTTP (HSTS, X-Frame-Options, CSP, X-Content-Type-Options).
9. Sesiones que no expiran por inactividad ni tienen tiempo máximo absoluto.
10. Usar el método GET para el formulario de login (credenciales en URL visible).
11. No proteger contra timing attacks (revelar si el usuario existe midiendo tiempos de respuesta).
12. No implementar logs de auditoría de autenticación para detectar ataques en tiempo real.

F.1 Preguntas de Síntesis

F.2.1 ¿Cuál es la diferencia esencial entre hash y cifrado? ¿Por qué se usa hash para contraseñas?

El cifrado es un proceso bidireccional: con la clave correcta se puede recuperar el dato original (ej: AES-256). El hash es unidireccional: no existe un proceso matemático para obtener el input original desde el output. Por eso se usa hash para contraseñas: el sistema no necesita 'conocer' la contraseña, solo verificar que su hash coincide con el almacenado. Si se cifraran, robar la clave de cifrado equivaldría a robar todas las contraseñas.

F.2.2 ¿Qué hace bcrypt diferente a SHA-256 para resistir ataques de fuerza bruta?

SHA-256 fue diseñado para ser lo más rápido posible (usado para verificar integridad de archivos grandes). Una GPU moderna puede calcular miles de millones de SHA-256 por segundo. bcrypt incorpora un factor de coste ajustable que hace que cada verificación tome deliberadamente ~100-300ms en hardware moderno; este tiempo se puede incrementar con el hardware futuro subiendo el factor. Lo que tarda segundos en un ataque SHA-256 tardaría siglos con bcrypt factor 12.

F.2.3 ¿Qué versión de TLS deben usar y por qué las anteriores están descartadas?

Deben usarse TLS 1.2 (mínimo) y preferiblemente TLS 1.3. SSL 2.0 y 3.0 tienen vulnerabilidades críticas (DROWN, POODLE). TLS 1.0 tiene BEAST y POODLE-TLS. TLS 1.1 tiene problemas con cipher suites débiles. El IETF los deprecó formalmente en RFC 8996 (2021). TLS 1.3 elimina todos esos problemas, hace PFS obligatorio y usa solo cipher suites modernos.

F.2.4 ¿Cuál fue el error más crítico del código CGI inseguro que analizamos hoy?

La SQL Injection mediante concatenación directa de variables en la query SQL: `sql = f"SELECT * FROM usuarios WHERE usuario='{usuario}' AND hash_password='{password}'"`. Este error permite a un atacante ingresar admin'-- en el campo usuario y bypassar completamente la verificación de contraseña, obteniendo acceso como administrador sin conocer ninguna credencial. Es el error más crítico porque compromete la autenticación completa.

F.2 Metacognición Personal

F.3.1 ¿Qué fue lo más sorprendente o revelador de la sesión de hoy?

La técnica del timing attack: el hecho de que no solo el contenido de la respuesta del servidor puede revelar información, sino el tiempo que tarda en responder. Si el servidor verifica la contraseña solo cuando el usuario existe, un atacante puede medir milisegundos de diferencia y construir una lista de usuarios válidos. El código seguro siempre ejecuta `bcrypt.checkpw()` con un hash dummy aunque el usuario no exista, para igualar los tiempos de respuesta.

F.3.2 ¿Qué concepto todavía no tienes completamente claro?

Perfect Forward Secrecy (PFS) y el intercambio de claves Diffie-Hellman efímero. Es claro que protege el tráfico pasado, pero el mecanismo matemático de cómo dos partes establecen una clave compartida sin transmitirla explícitamente por un canal inseguro requiere profundizar más en criptografía de clave pública.

F.3.3 ¿Qué error de seguridad en código hoy reconoces que podrías haber cometido (o has cometido) antes de esta clase?

Casi con certeza hubiera usado SHA-256 con sal para almacenar contraseñas, creyendo que era suficientemente seguro. Antes de conocer `bcrypt` y el concepto de 'hash lento de propósito específico', parecía razonable: es criptográficamente fuerte, usa sal, ¿qué podría fallar? La sesión dejó claro que la velocidad de SHA-256 es exactamente el problema.

F.3.4 ¿Cómo cambiaría tu forma de implementar un login después de esta sesión?

Implementaría `bcrypt` desde el inicio, usaría prepared statements sin excepción, añadiría rate limiting y bloqueo de cuenta desde el día 1, usaría mensajes de error genéricos invariablemente, registraría todos los intentos fallidos con IP y timestamp, y configuraría TLS 1.2+ con HSTS antes del primer despliegue. También añadiría MFA para sistemas con datos sensibles.

F.3 Tarea para la Semana 3 — Auditoría de login en campo

Sistema auditado: Portal del Estudiante — INTRANET Universidad Autónoma del Perú

Criterio de auditoría	Resultado	Herramienta usada
¿Usa HTTPS?	✓ Sí	Inspección de URL en navegador
¿Qué versión de TLS usa?	TLS 1.2 (debería ser 1.3)	SSL Labs / ssllabs.com/sslltest
¿Calificación SSL Labs?	B (por soporte de TLS 1.0 legacy)	ssllabs.com/sslltest
¿Formulario usa POST o GET?	POST (correcto)	Inspección de código fuente (F12)
¿Tiene política de bloqueo?	Sí, tras 10 intentos (valor alto)	Intento manual controlado
¿Ofrece MFA?	No (área de mejora crítica)	Revisión de opciones de la cuenta
¿Mensajes de error genéricos?	No: dice "Usuario no encontrado" separado de "Contraseña incorrecta"	Intento con usuarios inexistentes

Observación más importante:

El sistema muestra mensajes de error diferenciados ('Usuario no encontrado' vs 'Contraseña incorrecta'), lo que permite a un atacante enumerar usuarios válidos del sistema simplemente intentando logins con diferentes nombres de usuario.

Recomendación de mejora:

Unificar el mensaje de error a 'Credenciales incorrectas' en todos los casos, implementar MFA obligatorio para docentes y administrativos, actualizar la configuración TLS para deshabilitar TLS 1.0/1.1 y obtener calificación A en SSL Labs.
